Advanced React Concepts with File-Level Code Instructions

1. Introduction to useReducer

File: Counter.js

```
import React, { useReducer } from 'react';

const reducer = (state, action) => {
  switch (action.type) {
    case 'increment': return { count: state.count + 1 };
    case 'decrement': return { count: state.count - 1 };
    default: return state;
  }
};

export const Counter = () => {
  const [state, dispatch] = useReducer(reducer, { count: 0 });

  return (
    <div>
      <h2>Count: {state.count}</h2>
      <button onClick={() => dispatch({ type: 'increment' })}>+</button>
      <button onClick={() => dispatch({ type: 'decrement' })}>-</button>
    </div>
  );
};
```

2. useReducer vs useState

- useState: File: SimpleCounter.js
- useReducer: File: ComplexCounter.js

Use useState for primitive values like a counter.

Use useReducer for state logic involving multiple sub-values or actions.

3. Todo App with useReducer

File: TodoApp.js

```
import React, { useReducer, useState } from 'react';

const reducer = (todos, action) => {
  switch (action.type) {
    case 'add': return [...todos, { id: Date.now(), text: action.text }];
    case 'remove': return todos.filter(todo => todo.id !== action.id);
    default: return todos;
  }
};

export const TodoApp = () => {
  const [todos, dispatch] = useReducer(reducer, []);
  const [text, setText] = useState("");

  return (
    <div>
      <input value={text} onChange={(e) => setText(e.target.value)} />
      <button onClick={() => dispatch({ type: 'add', text })}>Add Todo</button>
      <ul>
        {todos.map(todo => (
          <li key={todo.id}>
            {todo.text}
            <button onClick={() => dispatch({ type: 'remove', id: todo.id })}>Remove</button>
          </li>
        ))}
      </ul>
    </div>
```

```
  );
};
```

4. useReducer Middleware

File: App.js

```
const middlewareDispatch = (action) => {
  console.log("Dispatching:", action);
  dispatch(action);
};
```

Use middlewareDispatch instead of dispatch for debugging or analytics.

5. Context + useReducer

Files:
- context/AppContext.js
- App.js

AppContext.js:

```
import { createContext, useReducer } from 'react';

const AppContext = createContext();
const reducer = (state, action) => { /*...*/ };
const AppProvider = ({ children }) => {
  const [state, dispatch] = useReducer(reducer, { user: null });
  return <AppContext.Provider value={{ state, dispatch }}>{children}</AppContext.Provider>;
};
export { AppContext, AppProvider };
```

App.js:

```
import { AppProvider } from './context/AppContext';
<AppProvider><App /></AppProvider>
```

6. Global State Management

Combine multiple contexts or use libraries like Redux Toolkit for scalable applications.

Create one context per domain (e.g., AuthContext, CartContext).

7. React.memo

File: MemoComponent.js

```
const Item = React.memo(({ value }) => <div>{value}</div>);
```

Use in parent: <Item value={item} />

8. useCallback and useMemo

File: OptimizedList.js

```
const filtered = useMemo(() => items.filter(i => i.active), [items]);
const handleClick = useCallback(() => doSomething(), [dependency]);
```

9. Lazy Loading

File: App.js

```
const LazyComponent = React.lazy(() => import('./LazyComponent'));
<Suspense fallback={<div>Loading...</div>}>
  <LazyComponent />
</Suspense>
```

10. Virtual DOM & Reconciliation

Use meaningful keys in lists (not index). Avoid unnecessary renders.

```
<ul>
  {items.map(item => <li key={item.id}>{item.name}</li>)}
```


11. Optimizing Product List

File: ProductList.js

- Use React.memo for list item components
- Use pagination
- useMemo for sorting/filtering

Example:

```
const sorted = useMemo(() => [...products].sort(), [products]);
```

12. React Profiler

Wrap component with Profiler for measurement:

```
<Profiler id="ProductList" onRender={(id, phase, actualDuration) => {
  console.log({ id, phase, actualDuration });
}}>
  <ProductList />
</Profiler>
```

Use DevTools > Profiler tab to inspect.